

# Synopses for the C library

Thomas Köppe <[tkoeppe@google.com](mailto:tkoeppe@google.com)>Richard Smith <[richard@metafoo.co.uk](mailto:richard@metafoo.co.uk)>

## Revision history

- 13 Feb 2016 (P0175r0): Initial proposal
- 24 Jun 2016 (P0175r1): this version: rebase on current working draft; replace `typedef` with `using`, add comments to mark which functions are newly added overloads; add missing `abs` overloads to `<cstdlib>` and `<cmath>`

## Contents

1. [Proposal](#)
2. [Impact on the standard](#)
3. [Proposed wording, Clause 18](#)
4. [Proposed wording, Clause 19](#)
5. [Proposed wording, Clause 20](#)
6. [Proposed wording, Clause 21](#)
7. [Proposed wording, Clause 22](#)
8. [Proposed wording, Clause 26](#)
9. [Proposed wording, Clause 27](#)

## Proposal

In this editorial paper we propose to add to the working draft the complete synopses of the C library headers that are included in C++ by reference to the C standard (see Table 15). These synopses will replace the various tables captioned “Header `<cfoo>` synopsis”.

The changes can roughly be summarized as follows.

- Each C header `<cfoo>` whose synopsis is being added gets its own subsection “[cfoo.syn]”. If necessary (usually that means that the synopsis says “see below”), the subsection starts with a new paragraph saying that the meaning of the contents of the header is the same as in C. Former “see also” references to the C standard may be moved into such a paragraph.
- The corresponding synopsis tables are deleted.
- Sections which describe multiple C headers may have their paragraphs moved into the appropriate new subsections.
- In two places ([c.strings] and [c.math]), wording that describes changes of signatures and addition of overloads is deleted and the overloads are directly given in the synopsis. The former wording is replaced with a note saying that the meaning of the functions is as in C, even though we have different signatures.

*Drafting note.* The synopses are largely copied from Annex B (library summary) of the C standard.

*Presentational note.* Insertions of entire code blocks are not underlined; the entire inserted block is decorated with vertical green rules.

*Drafting note.* Many of the existing “see also” references to the C standard appear to be incorrect (perhaps referring to the earlier C89).

## Impact on the standard

The change is purely editorial. Most internal references in the working draft are unaffected, with a few exceptions (e.g. references to `<cstdio>`).

## Proposed wording, Clause 18

In 18.2, rename the heading.

## 18.2 ~~Types~~Common definitions [support.types]

Delete Table 30 and paragraphs 1 and 2. Move the remaining content to new subsections as described below. Insert a new subsection.

### 18.2.x Header `<cstdint>` synopsis [cstdint.syn]

```
namespace std {
    using ptrdiff_t = see below;
    using size_t = see below;
    using max_align_t = see below;
    using nullptr_t = decltype(nullptr);
}

#define NULL see below

#define offsetof(P, D) see below
```

Insert a new subsection.

### 18.2.x Null pointers [support.types.nullptr]

Move the former paragraph 9 of 18.2 into this subsection with the following change.

~~nullptr\_t is defined as follows:~~

```
namespace std {
    using nullptr_t = decltype(nullptr);
}
```

The type ~~for which~~ `nullptr_t` is a synonym for the type of a `nullptr` expression, and it has the characteristics described in [...].

Move the former paragraph 3 of 18.2 into this subsection. Insert a new subsection.

### 18.2.x Sizes, alignments, and offsets [support.types.layout]

Move the former paragraphs 4 to 8 and the “see also” of 18.2 into this subsection, with the following change in the former paragraph 4.

[...] the ~~results are~~behavior is undefined [...]

In 18.3.1, change paragraph 1 as follows.

The headers `<limits>` (18.3.2), `<climits>` (18.3.3.1), and `<float>` (18.3.3.2) supply characteristics of implementation-dependent arithmetic types (3.9.1).

In 18.3.3, delete Tables 31 and 32 and all paragraphs. (Parts of paragraph 2 will be resurrected in the new section 18.3.3.1 below.) Insert a new subsection 18.3.3.1.

#### 18.3.3.1 Header `<climits>` synopsis [climits.syn]

```
#define CHAR_BIT see below
#define SCHAR_MIN see below
#define SCHAR_MAX see below
#define UCHAR_MAX see below
#define CHAR_MIN see below
```

```

#define CHAR_MAX see below
#define MB_LEN_MAX see below
#define SHRT_MIN see below
#define SHRT_MAX see below
#define USHRT_MAX see below
#define INT_MIN see below
#define INT_MAX see below
#define UINT_MAX see below
#define LONG_MIN see below
#define LONG_MAX see below
#define ULONG_MAX see below
#define LLONG_MIN see below
#define LLONG_MAX see below
#define ULLONG_MAX see below

```

1. The header `<climits>` defines all macros the same as 5.2.4.2.1 in the C standard. [Note: The types of the constants defined by macros in `<climits>` are not required to match the types to which the macros refer. – end note]

Drafting note. Check the C reference after the rebase onto C11.

Insert a new subsection 18.3.3.2.

### **18.3.3.2 Header `<float>` synopsis [`float.syn`]**

```

#define FLT_ROUNDS see below
#define FLT_EVAL_METHOD see below
#define FLT_HAS_SUBNORM see below
#define DBL_HAS_SUBNORM see below
#define LDBL_HAS_SUBNORM see below
#define FLT_RADIX see below
#define FLT_MANT_DIG see below
#define DBL_MANT_DIG see below
#define LDBL_MANT_DIG see below
#define FLT_DECIMAL_DIG see below
#define DBL_DECIMAL_DIG see below
#define LDBL_DECIMAL_DIG see below
#define DECIMAL_DIG see below
#define FLT_DIG see below
#define DBL_DIG see below
#define LDBL_DIG see below
#define FLT_MIN_EXP see below
#define DBL_MIN_EXP see below
#define LDBL_MIN_EXP see below
#define FLT_MIN_10_EXP see below
#define DBL_MIN_10_EXP see below
#define LDBL_MIN_10_EXP see below
#define FLT_MAX_EXP see below
#define DBL_MAX_EXP see below
#define LDBL_MAX_EXP see below
#define FLT_MAX_10_EXP see below
#define DBL_MAX_10_EXP see below
#define LDBL_MAX_10_EXP see below
#define FLT_MAX see below
#define DBL_MAX see below
#define LDBL_MAX see below
#define FLT_EPSILON see below
#define DBL_EPSILON see below
#define LDBL_EPSILON see below

```

```
#define FLT_MIN see below
#define DBL_MIN see below
#define LDBL_MIN see below
#define FLT_TRUE_MIN see below
#define DBL_TRUE_MIN see below
#define LDBL_TRUE_MIN see below
```

1. The header `<cfloat>` defines all macros the same as 5.2.4.2.2 in the C standard.

In 18.10, delete paragraph 2.

~~The contents of these headers are the same as the Standard C library headers `<setjmp.h>`, `<signal.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stdlib.h>`, and `<time.h>`, respectively, with the following changes:~~

Insert new subsections after paragraph 1 and move paragraphs 3, 4, 8 and 9 into the new subsections as described below. Insert a new subsection at the new end of 18.10.

### 18.10.x Header `<cstdarg>` synopsis [`cstdarg.h`]

```
namespace std {
    using va_list = see below;
}

#define va_arg(V, P) see below
#define va_copy(VDST, VSRC) see below
#define va_end(V) see below
#define va_start(V, P) see below
```

1. The contents of the header `<cstdarg>` are the same as the Standard C library header `<stdarg.h>`, with the following changes:

Append the former paragraph 3 of 18.10 to paragraph 1 above. Strike the subsequent “see also”. Insert a new subsection.

### 18.10.x Header `<csetjmp>` synopsis [`csetjmp.syn`]

```
namespace std {
    using jmp_buf = see below;
}

#define setjmp(env) see below

namespace std {
    [[noreturn]] void longjmp(jmp_buf env, int val);
}
```

1. The contents of the header `<csetjmp>` are the same as the Standard C library header `<setjmp.h>`.

*Drafting note.* Note the newly inserted `[[noreturn]]`.

Move the former paragraph 4 of 18.10 to this subsection. Strike the subsequent “see also”. Insert a new subsection.

### 18.10.x Header `<cstdbool>` synopsis [`cstdbool.syn`]

```
#define __bool_true_false_are_defined 1
```

1. The contents of the header <stdbool> are the same as the Standard C library header <stdbool.h>, with the following changes:

Append the former paragraph 8 of 18.10 to paragraph 1 above. Insert a new subsection.

### **18.10.x Header <csignal> synopsis [csignal.syn]**

```
namespace std {
    using sig_atomic_t = see below;
}

#define SIG_DFL see below
#define SIG_ERR see below
#define SIG_IGN see below
#define SIGABRT see below
#define SIGFPE see below
#define SIGILL see below
#define SIGINT see below
#define SIGSEGV see below
#define SIGTERM see below

namespace std {
    void (*signal(int sig, void (*func)(int)))(int);
    int raise(int sig);
}
```

1. The contents of the header <csignal> are the same as the Standard C library header <signal.h>.

Move the former paragraph 9 of 18.10 into this subsection. Insert a new subsection.

### **18.10.x Header <ctime> synopsis [ctime.syn]**

```
#define NULL see below
#define CLOCKS_PER_SEC see below
#define TIME_UTC see below

namespace std {
    using size_t = see below;
    using clock_t = see below;
    using time_t = see below;

    struct timespec;
    struct tm;

    clock_t clock(void);
    double difftime(time_t time1, time_t time0);
    time_t mktime(struct tm* timeptr);
    time_t time(time_t* timer);
    int timespec_get(timespec* ts, int base);
    char* asctime(const struct tm* timeptr);
    char* ctime(const time_t* timer);
    struct tm* gmtime(const time_t* timer);
    struct tm* localtime(const time_t* timer);
    size_t strftime(char* s, size_t maxsize, const char* format, const struct tm*
timeptr);
}
```

1. The contents of the header <ctime> are the same as the Standard C library header <time.h>.

# Proposed wording, Clause 19

Split section 19.3 [assertions]. Delete Table 42 and change paragraph 1:

The header `<cassert>`, ~~described in (Table 42),~~ provides a macro for documenting C++ program assertions and a mechanism for disabling the assertion checks.

After paragraph 1, insert a new section:

## 19.3.1 Header `<cassert>` synopsis [cassert.syn]

```
#define assert(E) see below
```

The former paragraphs 2 and 3 and the “see also” of 19. 3 become paragraphs 1 and 2 of a new subsection:

## 19.3.2 The `assert` macro [assertions.assert]

In 19.4 [errno], delete Table 43 and change paragraph 1 as follows.

~~The header `<cerrno>` is described in Table 43. Its contents~~The contents of the header `<cerrno>` are the same as the POSIX header `<errno.h>`, except that `errno` shall be defined as a macro. [...]

Insert a new section 19.4.1.

## 19.4.1 Header `<cerrno>` synopsis [cerrno.syn]

```
#define E2BIG see below
#define EACCES see below
#define EADDRINUSE see below
#define EADDRNOTAVAIL see below
#define EAFNOSUPPORT see below
#define EAGAIN see below
#define EALREADY see below
#define EBADF see below
#define EBADMSG see below
#define EBUSY see below
#define ECANCELED see below
#define ECHILD see below
#define ECONNABORTED see below
#define ECONNREFUSED see below
#define ECONNRESET see below
#define EDEADLK see below
#define EDESTADDRREQ see below
#define EDOM see below
#define EEXIST see below
#define EFAULT see below
#define EFBIG see below
#define EHOSTUNREACH see below
#define EIDRM see below
#define EILSEQ see below
#define EINPROGRESS see below
#define EINTR see below
#define EINVAL see below
#define EIO see below
#define EISCONN see below
#define EISDIR see below
#define ELOOP see below
#define EMFILE see below
#define EMLINK see below
```

```

#define EMSGSIZE see below
#define ENAMETOOLONG see below
#define ENETDOWN see below
#define ENETRESET see below
#define ENETUNREACH see below
#define ENFILE see below
#define ENOBUFS see below
#define ENODATA see below
#define ENODEV see below
#define ENOENT see below
#define ENOEXEC see below
#define ENOLCK see below
#define ENOLINK see below
#define ENOMEM see below
#define ENOMSG see below
#define ENOPROTOPT see below
#define ENOSPC see below
#define ENOSR see below
#define ENOSTR see below
#define ENOSYS see below
#define ENOTCONN see below
#define ENOTDIR see below
#define ENOTEMPTY see below
#define ENOTRECOVERABLE see below
#define ENOTSOCK see below
#define ENOTSUP see below
#define ENOTTY see below
#define ENXIO see below
#define EOPNOTSUPP see below
#define EOVERFLOW see below
#define EOWNERDEAD see below
#define EPERM see below
#define EPIPE see below
#define EPROTO see below
#define EPROTONOSUPPORT see below
#define EPROTOTYPE see below
#define ERANGE see below
#define EROFS see below
#define ESPIPE see below
#define ESRCH see below
#define ETIME see below
#define ETIMEDOUT see below
#define ETXTBSY see below
#define EWOULDBLOCK see below
#define EXDEV see below

#define errno see below

```

1. The meaning of the macros in this header is defined by the POSIX standard.

## Proposed wording, Clause 20

In 20.9.13 [c.malloc], remove paragraphs 1 and 2 and replace them with the following note.

[Note: The header <stdlib> (26.9.2) declares the functions `calloc`, `malloc`, `realloc` and `free`. – end note]

Delete Table 49 and remove the original paragraph 6.

~~Table 49 describes the header `<cstring>`.~~

Change the next paragraph as follows.

The contents [of the header `<cstring>` \(see 21.5.x\)](#) are the same as the Standard C library header `<string.h>`, with the change to `memchr()` specified in 21.5.

## Proposed wording, Clause 21

In 21.5 [c.strings], delete Tables 73, 74, 75, 76, 77, and 78 and remove paragraph 1.

~~Tables 73, 74, 75, 76, 77, and 78 describe headers `<cctype>`, `<cwctype>`, `<cstring>`, `<cwchar>`, `<cstdlib>` (character conversions), and `<cuchar>`, respectively.~~

Modify the next paragraph as follows.

The contents of ~~these headers~~ [the headers `<cctype>`, `<cwctype>`, `<cstring>`, `<cwchar>`, `<cstdlib>` \(see 26.9.2\), and `<cuchar>`](#) shall be the same as the Standard C Library headers `<ctype.h>`, `<wctype.h>`, `<string.h>`, `<wchar.h>`, and `<stdlib.h>` and the C Unicode TR header `<uchar.h>`, respectively, with the following modifications: [The functions `strchr`, `strpbrk`, `strchr`, `strstr`, `memchr`, `wcschr`, `wcspbrk`, `wcsrchr`, `wcsstr` and `wmemchr` have different signatures in this International Standard, but they shall have the same preconditions and the same behavior as in the C standard.](#)

Delete paragraphs 4 to 13 (deletions not shown here). Append a series of new subsections at the end of 21.5.

### [21.5.x Header `<cctype>` synopsis \[cctype.syn\]](#)

```
namespace std {
    int isalnum(int c);
    int isalpha(int c);
    int isblank(int c);
    int iscntrl(int c);
    int isdigit(int c);
    int isgraph(int c);
    int islower(int c);
    int isprint(int c);
    int ispunct(int c);
    int isspace(int c);
    int isupper(int c);
    int isxdigit(int c);
    int tolower(int c);
    int toupper(int c);
}
```

### [21.5.x Header `<cwctype>` synopsis \[cwctype.syn\]](#)

```
namespace std {
    using wint_t = see below;
    using wctrans_t = see below;
    using wctype_t = see below;
}

#define WEOF see below

namespace std {
    int iswalnum(wint_t wc);
    int iswalpha(wint_t wc);
    int iswblank(wint_t wc);
    int iswcntrl(wint_t wc);
```



```

int iswdigit(wint_t wc);
int iswgraph(wint_t wc);
int iswlower(wint_t wc);
int iswprint(wint_t wc);
int iswpunct(wint_t wc);
int iswspace(wint_t wc);
int iswupper(wint_t wc);
int iswxdigit(wint_t wc);
int iswctype(wint_t wc, wctype_t desc);
wctype_t wctype(const char* property);
wint_t towlower(wint_t wc);
wint_t towupper(wint_t wc);
wint_t towctrans(wint_t wc, wctrans_t desc);
wctrans_t wctrans(const char* property);
}

```

1. The meaning of the contents of this header is the same as the Standard C library header `<wctype.h>`.

## **21.5.x Header `<cstring>` synopsis [`cstring.syn`]**

```

namespace std {
    using size_t = see below;
}
#define NULL see below;

namespace std {
    void* memcpy(void* s1, const void* s2, size_t n);
    void* memmove(void* s1, const void* s2, size_t n);
    char* strcpy(char* s1, const char* s2);
    char* strncpy(char* s1, const char* s2, size_t n);
    char* strcat(char* s1, const char* s2);
    char* strncat(char* s1, const char* s2, size_t n);
    int memcmp(const void* s1, const void* s2, size_t n);
    int strcmp(const char* s1, const char* s2);
    int strcoll(const char* s1, const char* s2);
    int strncmp(const char* s1, const char* s2, size_t n);
    size_t strxfrm(char* s1, const char* s2, size_t n);
    const void* memchr(const void* s, int c, size_t n); // See 21.5
    void* memchr(void* s, int c, size_t n); // See 21.5
    const char* strchr(const char* s, int c); // See 21.5
    char* strchr(char* s, int c); // See 21.5
    size_t strcspn(const char* s1, const char* s2);
    const char* strpbrk(const char* s1, const char* s2); // See 21.5
    char* strpbrk(char* s1, const char* s2); // See 21.5
    const char* strrchr(const char* s, int c); // See 21.5
    char* strrchr(char* s, int c); // See 21.5
    size_t strspn(const char* s1, const char* s2);
    const char* strstr(const char* s1, const char* s2); // See 21.5
    char* strstr(char* s1, const char* s2); // See 21.5
    char* strtok(char* s1, const char* s2);
    void* memset(void* s, int c, size_t n);
    char* strerror(int errnum);
    size_t strlen(const char* s);
}

```

1. The meaning of the contents of this header is the same as the Standard C library header `<string.h>`.

## **21.5.x Header `<wchar>` synopsis [`wchar.syn`]**

```

namespace std {
    using wchar_t = see below;
    using size_t = see below;
    using mbstate_t = see below;
    using wint_t = see below;

    struct tm;
}

```

```

#define NULL see below
#define WCHAR_MAX see below
#define WCHAR_MIN see below
#define WEOF see below

```

```

namespace std {
    int fwprintf(FILE* stream, const wchar_t* format, ...);
    int fwsscanf(FILE* stream, const wchar_t* format, ...);
    int swprintf(wchar_t* s, size_t n, const wchar_t* format, ...);
    int swscanf(const wchar_t* s, const wchar_t* format, ...);
    int vfwprintf(FILE* stream, const wchar_t* format, va_list arg);
    int vfwscanf(FILE* stream, const wchar_t* format, va_list arg);
    int vswprintf(wchar_t* s, size_t n, const wchar_t* format, va_list arg);
    int vswscanf(const wchar_t* s, const wchar_t* format, va_list arg);
    int vwprintf(const wchar_t* format, va_list arg);
    int vwscanf(const wchar_t* format, va_list arg);
    int wprintf(const wchar_t* format, ...);
    int wscanf(const wchar_t* format, ...);
    wint_t fgetwc(FILE* stream);
    wchar_t* fgetws(wchar_t* s, int n, FILE* stream);
    wint_t fputwc(wchar_t c, FILE* stream);
    int fputws(const wchar_t* s, FILE* stream);
    int fwide(FILE* stream, int mode);
    wint_t getwc(FILE* stream);
    wint_t getwchar(void);
    wint_t putwc(wchar_t c, FILE* stream);
    wint_t putwchar(wchar_t c);
    wint_t ungetwc(wint_t c, FILE* stream);
    double wcstod(const wchar_t* nptr, wchar_t** endptr);
    float wcstof(const wchar_t* nptr, wchar_t** endptr);
    long double wcstold(const wchar_t* nptr, wchar_t** endptr);
    long int wcstol(const wchar_t* nptr, wchar_t** endptr, int base);
    long long int wcstoll(const wchar_t* nptr, wchar_t** endptr, int base);
    unsigned long int wcstoul(const wchar_t* nptr, wchar_t** endptr, int base);
    unsigned long long int wcstoull(const wchar_t* nptr, wchar_t** endptr, int base);
    wchar_t* wcsncpy(wchar_t* s1, const wchar_t* s2);
    wchar_t* wcsncpy(wchar_t* s1, const wchar_t* s2, size_t n);
    wchar_t* wmemcpy(wchar_t* s1, const wchar_t* s2, size_t n);
    wchar_t* wmemmove(wchar_t* s1, const wchar_t* s2, size_t n);
    wchar_t* wscat(wchar_t* s1, const wchar_t* s2);
    wchar_t* wcsncat(wchar_t* s1, const wchar_t* s2, size_t n);
    int wcscmp(const wchar_t* s1, const wchar_t* s2);
    int wscoll(const wchar_t* s1, const wchar_t* s2);
    int wcsncmp(const wchar_t* s1, const wchar_t* s2, size_t n);
    size_t wcsxfrm(wchar_t* s1, const wchar_t* s2, size_t n);
    int wmemcmp(const wchar_t* s1, const wchar_t* s2, size_t n);
    const wchar_t* wcschr(const wchar_t* s, wchar_t c); // See 21.5
    wchar_t* wcschr(wchar_t* s, wchar_t c); // See 21.5
    size_t wcsnspn(const wchar_t* s1, const wchar_t* s2);

```

```

const wchar_t* wcpbrk(const wchar_t* s1, const wchar_t* s2); // See 21.5
wchar_t* wcpbrk(wchar_t* s1, const wchar_t* s2); // See 21.5
const wchar_t* wcsrchr(const wchar_t* s, wchar_t c); // See 21.5
wchar_t* wcsrchr(wchar_t* s, wchar_t c); // See 21.5
size_t wcsspncpy(const wchar_t* s1, const wchar_t* s2);
const wchar_t* wcsstr(const wchar_t* s1, const wchar_t* s2); // See 21.5
wchar_t* wcsstr(wchar_t* s1, const wchar_t* s2); // See 21.5
wchar_t* wcstok(wchar_t* s1, const wchar_t* s2, wchar_t** ptr);
const wchar_t* wmemchr(const wchar_t* s, wchar_t c, size_t n); // See 21.5
wchar_t* wmemchr(wchar_t* s, wchar_t c, size_t n); // See 21.5
size_t wcslen(const wchar_t* s);
wchar_t* wmemset(wchar_t* s, wchar_t c, size_t n);
size_t wcsftime(wchar_t* s, size_t maxsize, const wchar_t* format, const struct tm*
timeptr);
wint_t btowc(int c);
int wctob(wint_t c);
int mbsinit(const mbstate_t* ps);
size_t mbrlen(const char* s, size_t n, mbstate_t* ps);
size_t mbrtowc(wchar_t* pwc, const char* s, size_t n, mbstate_t* ps);
size_t wctomb(char* s, wchar_t wc, mbstate_t* ps);
size_t mbsrtowcs(wchar_t* dst, const char** src, size_t len, mbstate_t* ps);
size_t wcsrtombs(char* dst, const wchar_t** src, size_t len, mbstate_t* ps);
}

```

1. The meaning of the contents of this header is the same as the Standard C library header `<wchar.h>`.

## 21.5.x Header `<cuchar>` synopsis [`cuchar.syn`]

```

namespace std {
    using mbstate_t = see below;
    using size_t = see below;

    size_t mbrtoc16(char16_t* pc16, const char* s, size_t n, mbstate_t* ps);
    size_t c16rtomb(char* s, char16_t c16, mbstate_t* ps);
    size_t mbrtoc32(char32_t* pc32, const char* s, size_t n, mbstate_t* ps);
    size_t c32rtomb(char* s, char32_t c32, mbstate_t* ps);
}

```

1. The meaning of the contents of this header is the same as the C Unicode TR header `<uchar.h>`.

## Proposed wording, Clause 22

In 22.6, delete Table 100 and paragraph 1.

~~Table 100 describes header `<locale>`.~~

Change paragraph 2 as follows.

The contents [of the header `<locale>`](#) are the same as the Standard C library header `<locale.h>`.

Strike the “see also”. Add a new subsection 22.6.1.

## Header `<locale>` synopsis [`locale.syn`]

```

namespace std {
    struct lconv;
}

```

```

#define NULL see below
#define LC_ALL see below
#define LC_COLLATE see below
#define LC_CTYPE see below
#define LC_MONETARY see below
#define LC_NUMERIC see below
#define LC_TIME see below

namespace std {
    char* setlocale(int category, const char* locale);

    lconv* localeconv();
}

```

1. The meaning of the content of this header is the same as in 7.11 in the C standard.

## Proposed wording, Clause 26

In 26.9, delete Tables 118 and 119 and paragraph 3.

~~Tables 118 and 119 describe headers `<cmath>` and `<cstdlib>`, respectively.~~

Change the former paragraph 4 as follows.

The contents of ~~these headers~~the headers `<cmath>` and `<cstdlib>` are the same as [...]

Delete paragraphs 6 to 9 (deletions not shown here), and in their stead insert a note.

[Note: Several functions have additional overloads in this International Standard, but they have the same behavior as in the C standard. – end note]

Delete paragraphs 11 and 12 (deletions not shown here).

From the former paragraph 14, delete the code listing (not shown fully).

[...] overloaded for the three floating-point types, ~~as follows:~~

*[delete remainder of paragraph]*

Change the former paragraph 15 as follows.

~~Moreover,~~For each set of overloaded functions within `<cmath>`, there shall be additional overloads [...]

Insert a new subsection 26.9.1.

### 26.9.1 Header `<cmath>` synopsis [`cmath.syn`]

```

namespace std {
    using float_t = see below;
    using double_t = see below;
}

#define HUGE_VAL see below
#define HUGE_VALF see below
#define HUGE_VALL see below
#define INFINITY see below
#define NAN see below
#define FP_INFINITE see below
#define FP_NAN see below
#define FP_NORMAL see below
#define FP_SUBNORMAL see below
#define FP_ZERO see below

```

```

#define FP_FAST_FMA see below
#define FP_FAST_FMAF see below
#define FP_FAST_FMAL see below
#define FP_ILOGB0 see below
#define FP_ILOGBNAN see below
#define MATH_ERRNO see below
#define MATH_ERREXCEPT see below

#define math_errhandling see below

namespace std {
    float acos(float x); // See 26.9
    double acos(double x);
    long double acos(long double x); // See 26.9
    float acosf(float x);
    long double acosl(long double x);

    float asin(float x); // See 26.9
    double asin(double x);
    long double asin(long double x); // See 26.9
    float asinf(float x);
    long double asinl(long double x);

    float atan(float x); // See 26.9
    double atan(double x);
    long double atan(long double x); // See 26.9
    float atanf(float x);
    long double atanl(long double x);

    float atan2(float y, float x); // See 26.9
    double atan2(double y, double x);
    long double atan2(long double y, long double x); // See 26.9
    float atan2f(float y, float x);
    long double atan2l(long double y, long double x);

    float cos(float x); // See 26.9
    double cos(double x);
    long double cos(long double x); // See 26.9
    float cosf(float x);
    long double cosl(long double x);

    float sin(float x); // See 26.9
    double sin(double x);
    long double sin(long double x); // See 26.9
    float sinf(float x);
    long double sinl(long double x);

    float tan(float x); // See 26.9
    double tan(double x);
    long double tan(long double x); // See 26.9
    float tanf(float x);
    long double tanl(long double x);

    float acosh(float x); // See 26.9
    double acosh(double x);
    long double acosh(long double x); // See 26.9
    float acoshf(float x);
    long double acoshl(long double x);

```

```
float asinh(float x); // See 26.9
double asinh(double x);
long double asinh(long double x); // See 26.9
float asinhf(float x);
long double asinhl(long double x);

float atanh(float x); // See 26.9
double atanh(double x);
long double atanh(long double x); // See 26.9
float atanhf(float x);
long double atanhhl(long double x);

float cosh(float x); // See 26.9
double cosh(double x);
long double cosh(long double x); // See 26.9
float coshf(float x);
long double coshl(long double x);

float sinh(float x); // See 26.9
double sinh(double x);
long double sinh(long double x); // See 26.9
float sinhlf(float x);
long double sinhl(long double x);

float tanh(float x); // See 26.9
double tanh(double x);
long double tanh(long double x); // See 26.9
float tanhf(float x);
long double tanhl(long double x);

float exp(float x); // See 26.9
double exp(double x);
long double exp(long double x); // See 26.9
float expf(float x);
long double expl(long double x);

float exp2(float x); // See 26.9
double exp2(double x);
long double exp2(long double x); // See 26.9
float exp2f(float x);
long double exp2l(long double x);

float expm1(float x); // See 26.9
double expm1(double x);
long double expm1(long double x); // See 26.9
float expm1f(float x);
long double expm1l(long double x);

float frexp(float value, int* exp); // See 26.9
double frexp(double value, int* exp);
long double frexp(long double value, int* exp); // See 26.9
float frexpf(float value, int* exp);
long double frexpl(long double value, int* exp);

int ilogb(float x); // See 26.9
int ilogb(double x);
int ilogb(long double x); // See 26.9
int ilogbf(float x);
int ilogbl(long double x);
```

```
float ldexp(float x, int exp); // See 26.9
double ldexp(double x, int exp);
long double ldexp(long double x, int exp); // See 26.9
float ldexpf(float x, int exp);
long double ldexpl(long double x, int exp);

float log(float x); // See 26.9
double log(double x);
long double log(long double x); // See 26.9
float logf(float x);
long double logl(long double x);

float log10(float x); // See 26.9
double log10(double x);
long double log10(long double x); // See 26.9
float log10f(float x);
long double log10l(long double x);

float log1p(float x); // See 26.9
double log1p(double x);
long double log1p(long double x); // See 26.9
float log1pf(float x);
long double log1pl(long double x);

float log2(float x); // See 26.9
double log2(double x);
long double log2(long double x); // See 26.9
float log2f(float x);
long double log2l(long double x);

float logb(float x); // See 26.9
double logb(double x);
long double logb(long double x); // See 26.9
float logbf(float x);
long double logbl(long double x);

float modf(float value, float* iptr); // See 26.9
double modf(double value, double* iptr);
long double modf(long double value, long double* iptr); // See 26.9
float modff(float value, float* iptr);
long double modfl(long double value, long double* iptr);

float scalbn(float x, int n); // See 26.9
double scalbn(double x, int n);
long double scalbn(long double x, int n); // See 26.9
float scalbnf(float x, int n);
long double scalbnl(long double x, int n);

float scalbln(float x, long int n); // See 26.9
double scalbln(double x, long int n);
long double scalbln(long double x, long int n); // See 26.9
float scalblnf(float x, long int n);
long double scalblnl(long double x, long int n);

float cbrt(float x); // See 26.9
double cbrt(double x);
long double cbrt(long double x); // See 26.9
float cbrtf(float x);
```

```
long double cbrtl(long double x);

int abs(int j); // See 26.9
long int abs(long int j); // See 26.9
long long int abs(long long int j); // See 26.9
float abs(float j); // See 26.9
double abs(double j);
long double abs(long double j); // See 26.9
float fabs(float x);
double fabs(double x);
long double fabs(long double x);
float fabsf(float x);
long double fabsl(long double x);

float hypot(float x, float y); // See 26.9
double hypot(double x, double y);
long double hypot(double x, double y); // See 26.9
float hypotf(float x, float y);
long double hypotl(long double x, long double y);

float hypot(float x, float y, float z); // See 26.9
double hypot(double x, double y, double z); // See 26.9
long double hypot(long double x, long double y, long double z); // See 26.9

float pow(float x, float y); // See 26.9
double pow(double x, double y);
long double pow(long double x, long double y); // See 26.9
float powf(float x, float y);
long double powl(long double x, long double y);

float sqrt(float x); // See 26.9
double sqrt(double x);
long double sqrt(long double x); // See 26.9
float sqrtf(float x);
long double sqrtl(long double x);

float erf(float x); // See 26.9
double erf(double x);
long double erf(long double x); // See 26.9
float erff(float x);
long double erfl(long double x);

float erfc(float x); // See 26.9
double erfc(double x);
long double erfc(long double x); // See 26.9
float erfcf(float x);
long double erfcl(long double x);

float lgamma(float x); // See 26.9
double lgamma(double x);
long double lgamma(long double x); // See 26.9
float lgammaf(float x);
long double lgammal(long double x);

float tgamma(float x); // See 26.9
double tgamma(double x);
long double tgamma(long double x); // See 26.9
float tgammaf(float x);
long double tgammal(long double x);
```



```
float ceil(float x); // See 26.9
double ceil(double x);
long double ceil(long double x); // See 26.9
float ceilf(float x);
long double ceill(long double x);

float floor(float x); // See 26.9
double floor(double x);
long double floor(long double x); // See 26.9
float floorf(float x);
long double floorl(long double x);

float nearbyint(float x); // See 26.9
double nearbyint(double x);
long double nearbyint(long double x); // See 26.9
float nearbyintf(float x);
long double nearbyintl(long double x);

float rint(float x); // See 26.9
double rint(double x);
long double rint(long double x); // See 26.9
float rintf(float x);
long double rintl(long double x);

long int lrint(float x); // See 26.9
long int lrint(double x);
long int lrint(long double x); // See 26.9
long int lrintf(float x);
long int lrintl(long double x);

long long int llrint(float x); // See 26.9
long long int llrint(double x);
long long int llrint(long double x); // See 26.9
long long int llrintf(float x);
long long int llrintl(long double x);

float round(float x); // See 26.9
double round(double x);
long double round(long double x); // See 26.9
float roundf(float x);
long double roundl(long double x);

long int lround(float x); // See 26.9
long int lround(double x);
long int lround(long double x); // See 26.9
long int lroundf(float x);
long int lroundl(long double x);

long long int llround(float x); // See 26.9
long long int llround(double x);
long long int llround(long double x); // See 26.9
long long int llroundf(float x);
long long int llroundl(long double x);

float trunc(float x); // See 26.9
double trunc(double x);
long double trunc(long double x); // See 26.9
float truncf(float x);
```

```
long double trunc1(long double x);

float fmod(float x, float y); // See 26.9
double fmod(double x, double y);
long double fmod(long double x, long double y); // See 26.9
float fmodf(float x, float y);
long double fmodl(long double x, long double y);

float remainder(float x, float y); // See 26.9
double remainder(double x, double y);
long double remainder(long double x, long double y); // See 26.9
float remainderf(float x, float y);
long double remainderl(long double x, long double y);

float remquo(float x, float y, int* quo); // See 26.9
double remquo(double x, double y, int* quo);
long double remquo(long double x, long double y, int* quo); // See 26.9
float remquof(float x, float y, int* quo);
long double remquol(long double x, long double y, int* quo);

float copysign(float x, float y); // See 26.9
double copysign(double x, double y);
long double copysign(long double x, long double y); // See 26.9
float copysignf(float x, float y);
long double copysignl(long double x, long double y);

double nan(const char* tagp);
float nanf(const char* tagp);
long double nanl(const char* tagp);

float nextafter(float x, float y); // See 26.9
double nextafter(double x, double y);
long double nextafter(long double x, long double y); // See 26.9
float nextafterf(float x, float y);
long double nextafterl(long double x, long double y);

float nexttoward(float x, long double y); // See 26.9
double nexttoward(double x, long double y);
long double nexttoward(long double x, long double y); // See 26.9
float nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);

float fdim(float x, float y); // See 26.9
double fdim(double x, double y);
long double fdim(long double x, long double y); // See 26.9
float fdimf(float x, float y);
long double fdiml(long double x, long double y);

float fmax(float x, float y); // See 26.9
double fmax(double x, double y);
long double fmax(long double x, long double y); // See 26.9
float fmaxf(float x, float y);
long double fmaxl(long double x, long double y);

float fmin(float x, float y); // See 26.9
double fmin(double x, double y);
long double fmin(long double x, long double y); // See 26.9
float fminf(float x, float y);
long double fminl(long double x, long double y);
```

```

float fma(float x, float y, float z); // See 26.9
double fma(double x, double y, double z);
long double fma(long double x, long double y, long double z); // See 26.9
float fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);

// Classification/comparison functions:
int fpclassify(float x);
int fpclassify(double x);
int fpclassify(long double x);

int isfinite(float x);
int isfinite(double x);
int isfinite(long double x);

int isinf(float x);
int isinf(double x);
int isinf(long double x);

int isnan(float x);
int isnan(double x);
int isnan(long double x);

int isnormal(float x);
int isnormal(double x);
int isnormal(long double x);

int signbit(float x);
int signbit(double x);
int signbit(long double x);

int isgreater(float x, float y);
int isgreater(double x, double y);
int isgreater(long double x, long double y);

int isgreaterequal(float x, float y);
int isgreaterequal(double x, double y);
int isgreaterequal(long double x, long double y);

int isless(float x, float y);
int isless(double x, double y);
int isless(long double x, long double y);

int islessequal(float x, float y);
int islessequal(double x, double y);
int islessequal(long double x, long double y);

int islessgreater(float x, float y);
int islessgreater(double x, double y);
int islessgreater(long double x, long double y);

int isunordered(float x, float y);
int isunordered(double x, double y);
int isunordered(long double x, long double y);
}

```

## 26.9.2 Header `<cstdlib>` synopsis [`cstdlib.syn`]

```

namespace std {
    using size_t = see below;
    using div_t = see below;
    using ldiv_t = see below;
    using lldiv_t = see below;
}

#define NULL see below
#define EXIT_FAILURE see below
#define EXIT_SUCCESS see below
#define RAND_MAX see below
#define MB_CUR_MAX see below

namespace std {
    double atof(const char* nptr);
    int atoi(const char* nptr);
    long int atol(const char* nptr);
    long long int atoll(const char* nptr);
    double strtod(const char* nptr, char** endptr);
    float strttof(const char* nptr, char** endptr);
    long double strtold(const char* nptr, char** endptr);
    long int strtol(const char* nptr, char** endptr, int base);
    long long int strtoll(const char* nptr, char** endptr, int base);
    unsigned long int strtoul(const char* nptr, char** endptr, int base);
    unsigned long long int strtoull(const char* nptr, char** endptr, int base);
    int rand(void);
    void srand(unsigned int seed);
    void* aligned_alloc(size_t alignment, size_t size);
    void* calloc(size_t nmemb, size_t size);
    void free(void* ptr);
    void* malloc(size_t size);
    void* realloc(void* ptr, size_t size);
    [[noreturn]] void abort(void);
    int atexit(void (*func)(void));
    int at_quick_exit(void (*func)(void));
    [[noreturn]] void exit(int status);
    [[noreturn]] void _Exit(int status);
    char* getenv(const char* name);
    [[noreturn]] void quick_exit(int status);
    int system(const char* string);

    extern "C" void* bsearch(const void* key, const void* base, size_t nmemb, size_t
size, int (*compar)(const void* , const void*)); // See 25.6
    extern "C++" void* bsearch(const void* key, const void* base, size_t nmemb, size_t
size, int (*compar)(const void* , const void*)); // See 25.6
    extern "C" void qsort(void* base, size_t nmemb, size_t size, int (*compar)(const
void* , const void*)); // See 25.6
    extern "C++" void qsort(void* base, size_t nmemb, size_t size, int (*compar)(const
void* , const void*)); // See 25.6

    int abs(int j);
    long int abs(long int j); // See 26.9
    long long int abs(long long int j); // See 26.9
    long int labs(long int j);

```

```

long long int llabs(long long int j);
float abs(float j); // See 26.9
double abs(double j); // See 26.9
long double abs(long double j); // See 26.9

div_t div(int numer, int denom);
ldiv_t div(long int numer, long int denom); // See 26.9
lldiv_t div(long long int numer, long long int denom); // See 26.9
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);

int mblen(const char* s, size_t n);
int mbtowc(wchar_t* pwc, const char* s, size_t n);
int wctomb(char* s, wchar_t wchar);
size_t mbstowcs(wchar_t* pwcs, const char* s, size_t n);
size_t wcstombs(char* s, const wchar_t* pwcs, size_t n);
}

```

1. The meaning of the contents of this header is the same as in 7.12 in the C standard.

## Proposed wording, Clause 27

In 27.11, delete Tables 151 and 152, and move all paragraphs as described below. Insert a new subsection 27.11.1.

### 27.11.1 Header <stdio> synopsis [stdio.syn]

```

namespace std {
    using size_t = see below;
    using FILE = see below;
    using fpos_t = see below;
}

#define NULL see below
#define _IOFBF see below
#define _IOLBF see below
#define _IONBF see below
#define BUFSIZ see below
#define EOF see below
#define FOPEN_MAX see below
#define FILENAME_MAX see below
#define L_tmpnam see below
#define SEEK_CUR see below
#define SEEK_END see below
#define SEEK_SET see below
#define TMP_MAX see below
#define stderr see below
#define stdin see below
#define stdout see below

namespace {
    int remove(const char* filename);
    int rename(const char* old, const char* new);
    FILE* tmpfile(void);
    char* tmpnam(char* s);
    int fclose(FILE* stream);
    int fflush(FILE* stream);
    FILE* fopen(const char* filename, const char* mode);
}

```

```

FILE* freopen(const char* filename, const char* mode, FILE* stream);
void setbuf(FILE* stream, char* buf);
int setvbuf(FILE* stream, char* buf, int mode, size_t size);
int fprintf(FILE* stream, const char* format, ...);
int fscanf(FILE* stream, const char* format, ...);
int printf(const char* format, ...);
int scanf(const char* format, ...);
int snprintf(char* s, size_t n, const char* format, ...);
int sprintf(char* s, const char* format, ...);
int sscanf(const char* s, const char* format, ...);
int vfprintf(FILE* stream, const char* format, va_list arg);
int vfscanf(FILE* stream, const char* format, va_list arg);
int vprintf(const char* format, va_list arg);
int vscanf(const char* format, va_list arg);
int vsnprintf(char* s, size_t n, const char* format, va_list arg);
int vsprintf(char* s, const char* format, va_list arg);
int vsscanf(const char* s, const char* format, va_list arg);
int fgetc(FILE* stream);
char* fgets(char* s, int n, FILE* stream);
int fputc(int c, FILE* stream);
int fputs(const char* s, FILE* stream);
int getc(FILE* stream);
int getchar(void);
int putc(int c, FILE* stream);
int putchar(int c);
int puts(const char* s);
int ungetc(int c, FILE* stream);
size_t fread(void* ptr, size_t size, size_t nmemb, FILE* stream);
size_t fwrite(const void* ptr, size_t size, size_t nmemb, FILE* stream);
int fgetpos(FILE* stream, fpos_t* pos);
int fseek(FILE* stream, long int offset, int whence);
int fsetpos(FILE* stream, const fpos_t* pos);
long int ftell(FILE* stream);
void rewind(FILE* stream);
void clearerr(FILE* stream);
int feof(FILE* stream);
int ferror(FILE* stream);
void perror(const char* s);
}

```

Move the former paragraph 1 of 27.11 into this subsection with the following change.

~~Table 151 describes header `<stdio>`.~~ The meaning of the content of this header is the same as in 7.19 in the C standard. [Note: C++ does not define the function `gets`. – end note]

Move the former paragraph 2 and the “see also” of 27.11 into this subsection. Insert a new subsection 27.11.2.

### 27.11.2 Header `<cinttypes>` synopsis [`cinttypes.syn`]

```

namespace std {
    using imaxdiv_t = see below;
}

#define PRIdN see below
#define PRIiN see below
#define PRIoN see below
#define PRIuN see below
#define PRIdN see below
#define PRiN see below

```

```
#define SCNdN see below
#define SCNiN see below
#define SCNoN see below
#define SCNuN see below
#define SCNxN see below
```

```
#define PRIdLEASTN see below
#define PRIiLEASTN see below
#define PRIoLEASTN see below
#define PRIuLEASTN see below
#define PRiXLEASTN see below
#define PRiXLEASTN see below
#define SCNdLEASTN see below
#define SCNiLEASTN see below
#define SCNoLEASTN see below
#define SCNuLEASTN see below
#define SCNxLEASTN see below
```

```
#define PRIdFASTN see below
#define PRIiFASTN see below
#define PRIoFASTN see below
#define PRIuFASTN see below
#define PRiXFASTN see below
#define PRiXFASTN see below
#define SCNdFASTN see below
#define SCNiFASTN see below
#define SCNoFASTN see below
#define SCNuFASTN see below
#define SCNxFASTN see below
```

```
#define PRIdMAX see below
#define PRIiMAX see below
#define PRIoMAX see below
#define PRIuMAX see below
#define PRiXMAX see below
#define PRiXMAX see below
#define SCNdMAX see below
#define SCNiMAX see below
#define SCNoMAX see below
#define SCNuMAX see below
#define SCNxMAX see below
```

```
#define PRIdPTR see below
#define PRIiPTR see below
#define PRIoPTR see below
#define PRIuPTR see below
#define PRiXPTR see below
#define PRiXPTR see below
#define SCNdPTR see below
#define SCNiPTR see below
#define SCNoPTR see below
#define SCNuPTR see below
#define SCNxPTR see below
```

```
namespace std {
    intmax_t imaxabs(intmax_t j);
    imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
    intmax_t strtointmax(const char* nptr, char** endptr, int base);
    uintmax_t strtouintmax(const char* nptr, char** endptr, int base);
```

```
intmax_t wcstoimax(const wchar_t* nptr, wchar_t** endptr, int base);
uintmax_t wcstoumax(const wchar_t* nptr, wchar_t** endptr, int base);
}
```

Move the former paragraph 3 of 27.11 to the new subsection with the following changes.

~~Table 152 describes header `<inttypes>`.~~ [Note: [...]]

Move the former paragraph 4 of 27.11 to the new subsection.

Update references of the form “`<stdio>` (27.11)” to say “27.11.1” in the following places: 27.4.1p1, 27.4.2p3,4,6, 27.4.3p3,4,6, 27.5.3.1.6p1, Footnote 335. Add the missing reference to 27.4.2p1 and 27.4.3p1.